

ComicCraft – AI Comic Story Creator using Gemini Model

Sub topic Frontend Development

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

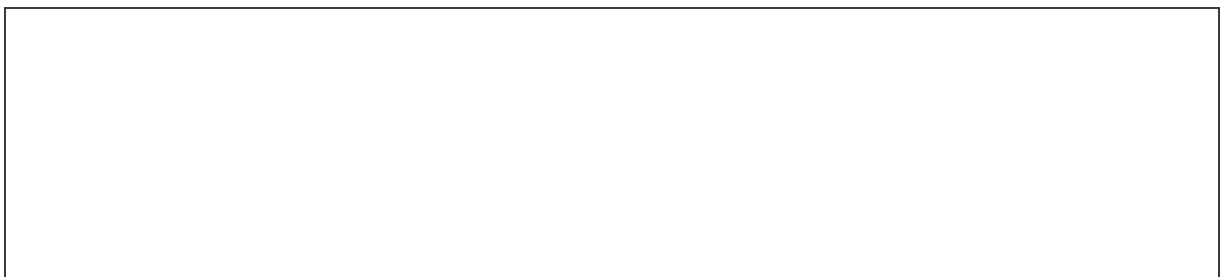
- Technology: Python
- Framework: FastAPI
- Template Engine: Jinja2
- Frontend: HTML + CSS
- AI Integration: Gemini Models
- Objective: Create dynamic web pages that display AI-generated comic content automatically.

Slide 2 — Introduction to Jinja2 Templates

What is Jinja2?

- Jinja2 is a Python-based **template engine** used to create dynamic HTML pages.
- FastAPI can use Jinja2 to render HTML templates.
- Data generated by Python can be passed directly to an HTML page.
- It helps separate **frontend design** from **backend application logic**.

ComicCraft Usage



Benefits:

- Dynamic content generation
- Reusable HTML templates

- Easy integration with Python
- Cleaner frontend development

Slide 3 — FastAPI and Jinja2 Configuration

Setting Up the Template Engine

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates

app = FastAPI()

templates = Jinja2Templates(
    directory="templates"
)

@app.get("/")
async def home(request: Request):
    return templates.TemplateResponse(
        "index.html",
        {"request": request}
    )
```

Project Structure



Jinja2 allows FastAPI to send Python data into the HTML templates.

Slide 4 — Creating Dynamic Comic Templates

Passing AI-Generated Data to Jinja2

FastAPI Code:

```
@app.get("/result")
async def result(request: Request):

    comic = {
```

```
        "title": "The Magic Adventure",
        "character": "Aria",
        "story": "Aria discovers a magical world."
    }

    return templates.TemplateResponse(
        "result.html",
        {
            "request": request,
            "comic": comic
        }
    )
```

Jinja2 HTML Template:

```
<h1>{{ comic.title }}</h1>

<h2>Character: {{ comic.character }}</h2>

<p>{{ comic.story }}</p>
```

Dynamic Rendering

- `{{ comic.title }}` displays the comic title.
- `{{ comic.character }}` displays character information.
- `{{ comic.story }}` displays the generated story.

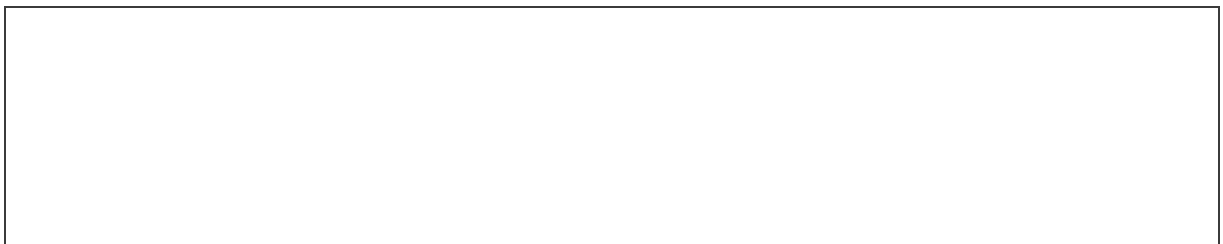
Slide 5 — Milestone 4 Outcome

Activity 4.2: Dynamic Template Development Completed

Key Achievements:

- Integrated **Jinja2** with FastAPI.
- Created reusable HTML templates.
- Passed Python data dynamically to webpages.
- Displayed Gemini-generated comic content using Jinja2.
- Created separate templates for input and result pages.
- Improved the flexibility and maintainability of the ComicCraft frontend.

Final Workflow





Outcome:

Dynamic Jinja2 templates successfully connect the **FastAPI backend** with the **ComicCraft frontend**, enabling AI-generated comic stories to be displayed dynamically.